Transfer or Self-Supervise? Label-Efficient Rooftop Segmentation for Solar Capacity Estimation

ECE 661 Course Project Proposal — Project D Methods on Overhead Imagery

Bruce Chen • Yachen He
Team 26, Duke University

Abstract

We propose to apply the transfer-learning methodology of Project D to rooftop segmentation in overhead imagery, as a step toward estimating potential rooftop solar capacity. On the SpaceNet 2 Las Vegas building-footprint dataset, we will evaluate pretrained MMSegmentation models without fine-tuning and fine-tune the most suitable one. We will then compare five initializations of the same encoder—random weights, driving-scene and remote-sensing checkpoints, and SimCLR or RotNet pretraining on unlabeled training imagery—at three label budgets, and carry segmentation errors through to roof area and an area-based capacity estimate. The central question is which source of pretrained features helps most when roof masks are scarce, at what cost, and whether the differences matter for the capacity estimate. Table 1 summarizes the proposal along the Heilmeier Catechism.

Heilmeier question	**Short answer**	**Sec.**
Q1. What are we trying to do?	Outline roofs in satellite images from few labeled examples; find which pretraining helps most and how errors change a solar-capacity estimate.	1
Q2. How is it done today? Limits?	Supervised networks need many labels per city; checkpoints come from other domains; unlabeled imagery goes unused; errors are rarely given in m^2 or kWp.	2
Q3. What is new; why will it work?	Project D's pipeline plus a controlled comparison: five encoder initializations $\times$ three label budgets, evaluated down to area and capacity error.	3
Q4. Who cares?	Planners and researchers estimating rooftop solar potential where labels are scarce.	4
Q5. What are the risks?	Fragile software stack, footprint-versus-roof mismatch, geographic leakage, unhelpful pretraining; each has a mitigation.	5
Q6. How much will it cost?	$\approx$34 TPU v5e chip-hours ($\approx$\$41 of our \$150 TPU share) and $\approx$68 Colab A100-hours ($\approx$\$37, outside the course credit), incl. 100% contingency.	6
Q7. How long will it take?	Five weeks; midterm check-in at the end of Week 3.	7
Q8. Mid-term and final "exams"?	Midterm: verified data, zero-shot results, baseline IoU ≥ 0.70, pretrained encoders. Final: the 5×3 comparison with IoU, area and capacity error, and cost.	8

Table 1: The proposal at a glance, following the Heilmeier Catechism.

1 What Are We Trying to Do? (Problem and Objectives) Heilmeier Q1

1.1 The problem in plain language

To know how much solar power a city could install on its rooftops, we first need to know how much roof area it has. Satellite images show every roof, but turning them into roof outlines by hand is slow, so we would like a computer to do it. A segmentation network can learn to outline roofs, but only from examples that people have traced, and such examples exist for only a few cities. Ready-made pretrained networks are available, yet they learned from other kinds of pictures: street scenes photographed from a car, or aerial images taken at a very different resolution or in a different country. Meanwhile, untraced satellite images of the same city are plentiful. We want to find out which starting point lets a network outline roofs best from only a handful of traced examples, how much that starting point costs to prepare, and whether the choice changes the solar-capacity number a planner would read.

1.2 Formal problem statement

Let $\mathcal{D}_L = \{(x^{(i)}, y^{(i)})\}$ be the labeled image chips of the training partition, with pan-sharpened RGB images $x \in [0,1]^{3\times H\times W}$ at 0.3 m ground sample distance and masks $y \in \{0,1\}^{H\times W}$, where 1 marks pixels inside a labeled building footprint, our roof-area proxy. Let $\mathcal{D}_U$ contain the same training images without labels. A segmentation network $s = g_\phi \circ f_\theta$ consists of a ResNet-50 encoder f_θ and a DeepLabV3+ decoder g_ϕ. The encoder starts from

$$\theta_0 \in \Theta_0 = \left\{\theta_{\text{rand}},\ \theta_{\text{city}},\ \theta_{\text{RS}},\ \theta_{\text{SimCLR}}(\mathcal{D}_U),\ \theta_{\text{Rot}}(\mathcal{D}_U)\right\},$$

where θ_{city} and θ_{RS} are taken from MMSegmentation checkpoints trained on driving scenes and on remote-sensing imagery. The network is fine-tuned on a subset $\mathcal{D}_L^{(n)} \subseteq \mathcal{D}_L$ containing a fraction $n \in \{5\%, 20\%, 100\%\}$ of the labeled chips. We seek how roof IoU on geographically held-out test areas, and the area and capacity errors of Section 3.5, depend on (θ_0, n), subject to pretraining within our $150 TPU share and fine-tuning within a modest Colab budget.

1.3 Objectives

- **O1. Data pipeline (Project D basic).** Download SpaceNet 2 Las Vegas, rasterize the footprint polygons into masks in MMSegmentation format, split by geographic area, visualize samples, document core statistics (number of chips, resolution, roof-pixel fraction), and verify the pixel-to-m^2 conversion.
- **O2. Transfer-learning baseline (Project D basic).** Evaluate at least two pretrained MMSegmentation models on the data without fine-tuning, then fine-tune the most suitable one, targeting IoU, precision and recall.
- **O3. Controlled initialization comparison (our extension).** Compare five encoder initializations at three label budgets on one fixed architecture, adapting the self-supervised comparison of Project 4 from classification to segmentation.
- **O4. Area, capacity and cost.** Convert predicted masks into roof area (m^2) and estimated potential capacity (kWp), measure how segmentation errors propagate into both, and report pretraining and fine-tuning compute separately.
- **O5. Reproducible deliverables.** Code, configuration files, training logs, a README with exact commands, figures, the poster and the final report.

Because this changes the listed project's task and dataset, we will seek TA approval for the adapted scope before Week 2.

2 How Is It Done Today, and What Are the Limits? Heilmeier Q2

Rooftop solar potential. National estimates such as NREL's rooftop technical-potential study derive suitable roof planes from LiDAR, then convert roof area to capacity with an assumed module power density and packing factor [1]. Deep learning on satellite imagery has been used at national scale to map solar installations that already exist [2]. LiDAR coverage is uneven, however, and imagery-based methods need labeled examples.

Building segmentation from overhead imagery. The SpaceNet challenges released building-footprint labels for several cities and established fully convolutional networks [5, 6] as the standard approach [3]. These models are trained with thousands of labeled chips per city.

Transfer learning. When labels are few, a common remedy is to start from a pretrained model. MMSegmentation [7] provides checkpoints of the same DeepLabV3+ architecture [8] trained on driving scenes (Cityscapes [10]), on satellite land cover (LoveDA [11]) and on aerial imagery (ISPRS Potsdam [12]). Each differs from our target in viewpoint, resolution or geography.

Self-supervised pretraining. SimCLR [13] learns features by making two augmented views of the same image agree; RotNet [14] learns them by predicting which rotation was applied. Remote-sensing variants exist, for example contrastive learning across seasons [15], but most evaluations target classification or land cover rather than fine building masks.

Limits of current practice relevant to us. (1) *Labels are costly and region-specific:* footprint masks exist for few cities. (2) *Domain gap:* available checkpoints were trained on street views, other resolutions or other countries. (3) *Unused unlabeled imagery:* untraced images of the target city are plentiful but ignored by supervised pipelines. (4) *Pixel metrics versus estimates:* IoU does not say how wrong a derived roof area or capacity estimate is, which is what a planner would read.

3 What Is New, and Why Will It Succeed? (Methodology) Heilmeier Q3

Figure 1 summarizes the system. All arms share one architecture, one labeled dataset, one fine-tuning procedure and one evaluation protocol, so that observed differences can be attributed to the encoder initialization θ_0 and the label budget n.

3.1 Data, labels and geographic split

SpaceNet 2 [3] provides WorldView-3 imagery with building-footprint polygons for four cities under a CC BY-SA 4.0 license; the Las Vegas area covers 216 km^2 with about 151,000 labeled buildings. Images are cut into 650×650-pixel chips of 200×200 m, and the pan-sharpened RGB product has a 0.3 m ground sample distance. Using publicly available SpaceNet utilities, as Project D suggests for its own dataset, we will rasterize the polygons into binary masks in MMSegmentation format and rescale the 16-bit imagery to 8 bits with a fixed percentile stretch computed on the training partition. SpaceNet labels are building *footprints*, not visible or usable roof area, so we will call our measurements roof-area proxies and limit our claims accordingly.

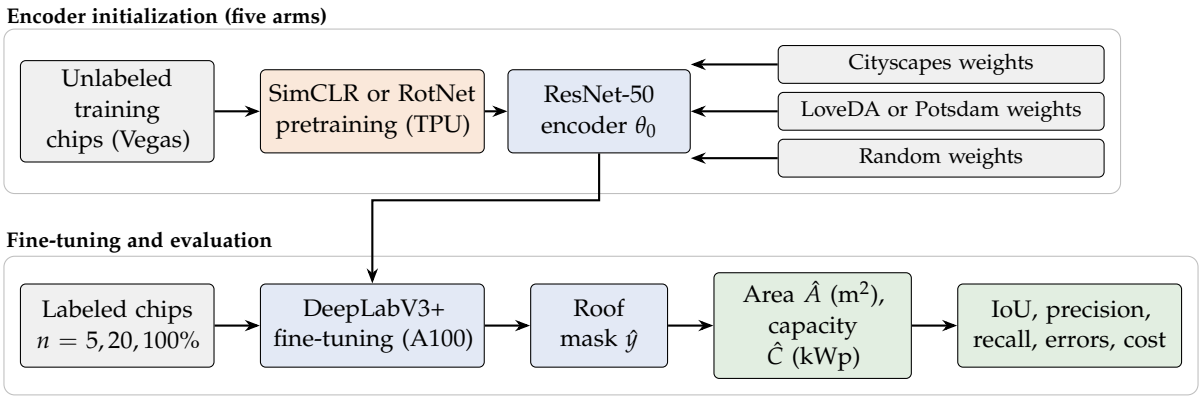


Figure 1: System overview. Top: the five encoder initializations; only SimCLR and RotNet use the unlabeled training imagery. Bottom: every arm is fine-tuned with the same labeled chips, schedule and decoder, then evaluated on pixels, on derived area and capacity, and on cost. Only θ_0 and n vary across runs.

Before extracting any chip, we will divide Las Vegas into contiguous geographic blocks of about 1 km^2 and assign whole blocks to training, validation and test partitions (roughly 70/15/15), leaving out a one-chip buffer between partitions so that adjacent chips never fall on different sides of a split. Self-supervised pretraining uses only training-partition images. Each test block serves as one evaluation area for the area and capacity estimates. To convert pixels to square meters, we will read each chip's georeferencing, reproject to the local UTM zone if the chips are stored in geographic coordinates, and check the result against footprint areas computed directly from the polygons.

3.2 Project D baseline: pretrained models without and with fine-tuning

Following Project D, we take three DeepLabV3+ R-50 checkpoints from the MMSegmentation model zoo, which share an architecture but were trained on different domains: Cityscapes (street-level driving scenes), LoveDA (satellite land cover at 0.3 m) and Potsdam (aerial imagery at about 5 cm). All three label spaces contain a "building" class, so we can evaluate them without any fine-tuning by mapping that class to our roof mask. For Potsdam we will also test rescaling the input to reduce the resolution mismatch. We then fine-tune the most promising checkpoint on all training chips, keeping its decoder and replacing only the final classification layer, and report IoU, precision and recall on the validation blocks. These runs also fix the training settings in Table 2.

3.3 Encoder initializations

All five arms use the ResNet-50 encoder of DeepLabV3+; only its starting weights θ_0 differ.

Random. Standard random initialization; the purely supervised reference point.

Cityscapes. Encoder weights of the Cityscapes checkpoint: the out-of-domain transfer route of Project D.

Remote sensing. Encoder weights of whichever of the LoveDA and Potsdam checkpoints performs better without fine-tuning on the validation blocks: a near-domain transfer route.

SimCLR. Two augmented views $\tilde{x}_i, \tilde{x}_j$ of the same unlabeled chip are encoded and projected to $z = h(f_\theta(\tilde{x}))$. With $2N$ views per batch and cosine similarity sim, the NT-Xent loss [13] for a positive pair is

$$\ell_{i,j} = -\log \frac{\exp\big(\mathrm{sim}(z_i, z_j)/\tau\big)}{\sum_{k \neq i} \exp\big(\mathrm{sim}(z_i, z_k)/\tau\big)}, \tag{1}$$

Setting	Value
Data	SpaceNet 2 Las Vegas: WorldView-3 pan-sharpened RGB, 0.3 m GSD, 650×650 chips (200×200 m); footprint polygons rasterized to masks; 16-bit to 8-bit percentile stretch
Classes	2: background and roof (building-footprint proxy)
Split	Geographic blocks of $\approx$1 km^2, $\approx$70/15/15 train/validation/test, one-chip buffer between partitions; each test block is one evaluation area
Label budgets	$n \in \{5\%, 20\%, 100\%\}$ of training chips, identical across arms; 3 random subsets (seeds) for $n \in \{5\%, 20\%\}$ (E3)
Unlabeled pool	All training-partition chips with labels removed
Architecture	DeepLabV3+ [8], ResNet-50 encoder [9] (R-50-D8, output stride 8); decoder newly initialized in every E2 arm
Project D models	DeepLabV3+ R-50-D8 checkpoints trained on Cityscapes, LoveDA and Potsdam, from the MMSegmentation model zoo
SimCLR	224×224 crops, batch 128 pairs, temperature $\tau = 0.2$, 2-layer projection head (128-d), SGD with cosine schedule, 200 epochs, bf16
RotNet	Rotations $\{0^\circ, 90^\circ, 180^\circ, 270^\circ\}$, linear rotation head, batch 256, 100 epochs
Augmentations	Random resized crop, flips, 90° rotations (not in the RotNet pretext), color jitter, Gaussian blur
Fine-tuning	SGD (lr 0.01, momentum 0.9, weight decay 5×10^{-4}), poly decay, 4k iterations, batch 8, 512×512 crops, scale 0.75–1.25, mixed precision; cross-entropy loss
Probe	Encoder frozen; decoder trained with the fine-tuning schedule
Capacity constants	Usable fraction $u = 0.70$ and power density $\rho = 0.16$ kWp/m^2, after [1]; fixed for all models
Hardware	Colab A100 (all MMSegmentation runs); one Cloud TPU v5e chip with PyTorch/XLA (pretraining)

Table 2: Planned settings (subject to tuning during Weeks 1–2; final values will be documented).

averaged over all positive pairs. The projection head h is discarded after pretraining.

RotNet. Each unlabeled chip is rotated by $r \in \mathcal{G} = \{0^\circ, 90^\circ, 180^\circ, 270^\circ\}$ and a linear head p_ψ predicts r [14]:

$$\mathcal{L}_{\text{rot}}(\theta, \psi) = -\frac{1}{|\mathcal{G}|} \sum_{r \in \mathcal{G}} \log p_\psi\big(r \mid f_\theta(\text{Rot}(x, r))\big). \tag{2}$$

Both self-supervised methods pretrain exactly the ResNet-50 definition used by the segmentation encoder, so the weights load without modification.

3.4 Fine-tuning at controlled label budgets

For each θ_0 and budget n, the decoder is newly initialized and the whole network is trained on $\mathcal{D}_L^{(n)}$ with pixel-wise cross-entropy,

$$\mathcal{L}_{\text{seg}} = -\frac{1}{|\Omega|} \sum_{q \in \Omega} \log s(x)_{q, y_q}, \tag{3}$$

where Ω is the set of pixels in a crop. All arms at a given budget receive the same chips, schedule, augmentation and model selection on the validation blocks; we choose among a small set of settings rather than run a large sweep. For each pretrained arm we also run a *probe*: the encoder is frozen and only the decoder is trained. This segmentation-oriented probe plays the role that linear evaluation plays for classification in Project 4.

3.5 From segmentation to area and capacity

For evaluation area k with pixel set $\mathcal{P}_k$, where pixel q covers a_q square meters on the ground,

$$\hat{A}_k = \sum_{q \in \mathcal{P}_k} a_q \, \mathbf{1}\big[\hat{y}_q = 1\big], \qquad \hat{C}_k = u \, \rho \, \hat{A}_k, \tag{4}$$

where $\hat{A}_k$ is the predicted roof-proxy area, $\hat{C}_k$ the estimated potential capacity in kWp, u an assumed usable fraction and ρ an assumed installed power density. The same formulas applied to the reference masks give A_k and C_k. We report

$$e_k = \hat{A}_k - A_k, \qquad r_k = \frac{\hat{A}_k - A_k}{A_k} = \frac{\hat{C}_k - C_k}{C_k}, \tag{5}$$

as signed and absolute values per evaluation area and in aggregate, together with the capacity error in kWp. Because $\hat{C}_k$ is linear in $\hat{A}_k$, the relative capacity error equals the relative area error: changing u or ρ rescales every model's capacity error equally and is never treated as evidence that one model is better. The capacity values are estimates under stated assumptions; they do not establish whether a roof is structurally suitable for solar installation or predict electricity generation.

3.6 Evaluation protocol

- **Segmentation quality:** roof IoU, precision and recall on the geographically held-out test blocks.
- **Area and capacity error:** signed and absolute e_k (m^2), r_k (%) and capacity error (kWp), per evaluation area and in aggregate.
- **Representation quality:** probe IoU for each pretrained arm; the RotNet pretext accuracy.
- **Uncertainty:** 95% bootstrap intervals over test chips; the spread across labeled subsets (E3).
- **Cost:** pretraining and fine-tuning hours by hardware, reported separately, and inference time per km^2.

3.7 Experiments

E1. Project D baseline: the Cityscapes, LoveDA and Potsdam checkpoints evaluated without fine-tuning; full fine-tuning of the most promising one at $n = 100\%$.
E2. Initialization $\times$ budget: the five arms at $n \in \{5\%, 20\%, 100\%\}$ (15 runs).
E3. Repeated subsets: the five arms at $n \in \{5\%, 20\%\}$ with two additional random labeled subsets (20 runs).
E4. Probes: frozen-encoder probes for the four pretrained arms at each budget (12 runs).
E5. Area and capacity error: e_k, r_k and capacity error for all E2 models.
E6. Cost: compute and wall-clock time for every pretraining and fine-tuning run.

Stretch goals (only if ahead of schedule): cross-region evaluation on another SpaceNet 2 city without tuning on its test labels; a cross-entropy plus Dice loss to sharpen roof boundaries, one of the improvements Project D suggests; continued SimCLR pretraining starting from the remote-sensing weights.

3.8 What is new, and why we expect it to succeed

New. (1) Project D's transfer pipeline applied to overhead imagery and extended into a single controlled comparison of out-of-domain transfer, near-domain transfer, in-domain self-supervision and no pretraining, across label budgets, on one architecture. (2) Segmentation errors carried through to roof area and potential capacity, rather than reported as IoU alone. (3) Accuracy reported together with the compute each route costs.

Why it will work. Every component has a public reference implementation: MMSegmentation provides checkpoints of the same DeepLabV3+ R-50 model for all three source domains, and SimCLR and RotNet have standard recipes. Each fine-tuning run takes roughly half an A100-hour, so the full grid is affordable. We de-risk by pinning the software environment in Week 1, checking masks and the area conversion against the polygons before any training, and starting pretraining in Week 2 so that one rerun still fits.

4 Who Cares? If We Succeed, What Difference Will It Make? Heilmeier Q4

- **Planners and researchers estimating rooftop solar potential.** Many regions lack LiDAR and labeled footprints. Knowing whether limited effort is better spent on reusing existing checkpoints, on self-supervised pretraining with local imagery, or on labeling more chips is directly actionable.
- **Decision-relevant evaluation.** Reporting errors in m^2 and kWp per area shows whether an IoU gain changes the estimate a planner would use, and whether errors cancel in aggregate or bias it.
- **Course practice.** Project D targets the skill of adapting an existing toolbox and its pretrained models to a new domain. A documented, reproducible MMSegmentation pipeline for SpaceNet, plus a clear account of what transfers and what does not, serves that goal.

5 What Are the Risks? Heilmeier Q5

Table 3 lists the main risks, how likely we judge each one to be, and how we will mitigate it. The budget-related risks are backed by the spending rules in Section 6.

6 How Much Will It Cost? Heilmeier Q6

The project needs no paid data or software: SpaceNet 2 is public under CC BY-SA 4.0 and can be downloaded anonymously from its AWS Open Data bucket [4], and the code uses open-source PyTorch, PyTorch/XLA and MMSegmentation. The monetary cost is compute. The course provides each group up to $250 of Google Cloud credit, split between a shared Gemini API budget, which each member accesses with their own course key, and TPU compute, which runs in Cloud TPU sessions started with the course `ece661` tool. We have allocated $100 to the Gemini API, which we use for coding assistance, and $150 to TPU compute.

Because MMSegmentation relies on CUDA operators, all segmentation training and evaluation will run on Colab A100 GPUs, paid with Colab compute units outside the course credit. Self-supervised pretraining is plain PyTorch and runs on the TPU. Human effort is about 10–12 hours per member per week, roughly 110 person-hours in total.

Table 4 prices TPU time at the on-demand TPU v5e rate of $1.20 per chip-hour in US regions [16] and Colab time at about 5.4 compute units ($0.54) per A100 (40 GB) hour [17]. Per-run

Risk	Likelihood	Mitigation
MMSegmentation environment breaks on Colab (MMCV <2.2.0 needs matching PyTorch/CUDA builds)	Medium	Pin a working PyTorch/CUDA/MMCV combination in Week 1 and save the setup script; if still unstable, implement the same DeepLabV3+ in plain PyTorch with converted weights, after consulting the TA.
PyTorch/XLA problems on TPU (recompilation, slow input pipeline)	Medium	Static shapes (fixed crop size, `drop_last`); augmentation on host CPU; TPU smoke test in Week 1; fall back to shorter pretraining on Colab A100.
Footprints differ from visible or usable roof area	High	Call all measurements roof-area proxies; inspect overlays for misalignment; analyze failure cases; make no claims about usable roof area.
Geographic leakage between partitions	Medium	Split by $\approx$1 km^2 blocks with a one-chip buffer before cropping; versioned split file checked by script; pretraining uses training-partition images only.
Wrong pixel-to-m^2 conversion	Low	Reproject to UTM; compare rasterized mask areas with polygon areas on sample chips before any training.
Large download (Las Vegas archive $\approx$24 GB), 16-bit imagery	Medium	Download once to Colab, keep only pan-sharpened RGB and labels on Google Drive; one fixed percentile stretch for all arms.
Pretraining does not help; RotNet exploits shadow direction	Medium	Probes (E4) diagnose representation quality; negative results are reported as findings.
Colab session disconnects or no A100 available	Medium	Checkpoint every 500 iterations to Google Drive; runs are short; fall back to an L4 GPU at lower throughput.
Overspending	Low	Debug on CPU or a free Colab T4 first; end sessions promptly; weekly spend check (Section 6).
Two-person team (lower capacity)	Medium	Scope limited to Project D's basic requirements plus one extension; stretch goals explicitly optional; balanced ownership (Table 5).

Table 3: Risks and mitigations.

times are conservative estimates (about 0.5 A100-hours per 4k-iteration fine-tuning run); we will measure actual throughput in Week 1 and revise the plan.

Spending rules. (1) TPU sessions and Colab runtimes are ended as soon as a job finishes, because idle hardware keeps billing; a v5e chip left running over a 60-hour weekend would cost \$72.00, almost half of our TPU share. (2) All debugging happens on CPU or a free Colab T4 before any paid run. (3) We check spending at the end of every week; if TPU spending exceeds \$75 or Colab spending exceeds \$25 by the midterm check-in, the stretch goals are dropped. (4) The plan is robust to hardware: at the TPU v6e (Trillium) rate of \$2.70 per chip-hour [16], the 34 planned chip-hours would cost \$91.80, still within the TPU share; if only the 80 GB A100 is available ($\approx$\$0.75/hour), the Colab plan would cost about \$51. If the TPU route proves impractical in Week 1, we will pretrain on Colab with fewer epochs and ask a TA whether part of the TPU allocation should move, since changes after October 23 go through a TA.

Item	Platform	Hours	Cost (USD)
TPU smoke tests and throughput measurement	TPU v5e	2	$2.40
SimCLR pretraining (ResNet-50, 200 epochs)	TPU v5e	6	$7.20
RotNet pretraining (100 epochs)	TPU v5e	4	$4.80
Stretch pretraining (SimCLR from remote-sensing weights)	TPU v5e	5	$6.00
TPU subtotal		17	$20.40
Contingency (100%: reruns, failed jobs, idle time)		17	$20.40
TPU total planned		**34**	**$40.80**
Unallocated TPU reserve		—	$109.20
Environment setup, smoke tests, pilots	Colab A100	4	$2.16
E1: Project D baseline (3 zero-shot evaluations, 1 fine-tune)	Colab A100	2	$1.08
E2: initialization × budget (15 runs)	Colab A100	8	$4.32
E3: repeated labeled subsets (20 runs)	Colab A100	10	$5.40
E4: frozen-encoder probes (12 runs)	Colab A100	4	$2.16
E5–E6: inference, area and capacity, timing	Colab A100	2	$1.08
Stretch explorations	Colab A100	4	$2.16
Colab subtotal		34	$18.36
Contingency (100%)		34	$18.36
Colab total planned (outside course credit)		**68**	**$36.72**
Gemini API, coding assistance (shared course budget)	Course keys	—	≤$100.00
Course credit (Gemini API $100 + TPU $150)			$250.00

Table 4: Compute budget. TPU priced at the v5e on-demand rate of $1.20/chip-hour; Colab priced at the A100 (40 GB) rate of ≈$0.54/hour.

7 How Long Will It Take? (Timeline and Division of Labor) Heilmeier Q7

The project spans five weeks beginning the week of October 26 (Figure 2), with each week split into two halves and bars colored by owner. The midterm check-in with the TAs is at the end of Week 3, and the poster session and final report follow Week 5. Thanksgiving falls in the second half of Week 5, so all core experiments are scheduled to finish in Week 4.

Division of labor. Ownership is split so that each member leads one half of every stage and the workloads are comparable (Table 5). Both members review each other's code, take part in all experiments, and share the writing of the report and poster equally. We will review the balance at the midterm check-in, and we understand that a two-person team is not held to reduced basic project expectations.

8 What Are the Mid-Term and Final "Exams"? (Expected Outcomes) Heilmeier Q8

8.1 Expected outcomes and hypotheses

H1. Without fine-tuning (E1), the LoveDA checkpoint, which matches our 0.3 m resolution and overhead view, segments roofs far better than the street-level Cityscapes checkpoint; the 5 cm Potsdam checkpoint improves when the input is rescaled.

H2. Every pretrained initialization beats random initialization at $n = 5\%$, and the gap narrows at $n = 100\%$.

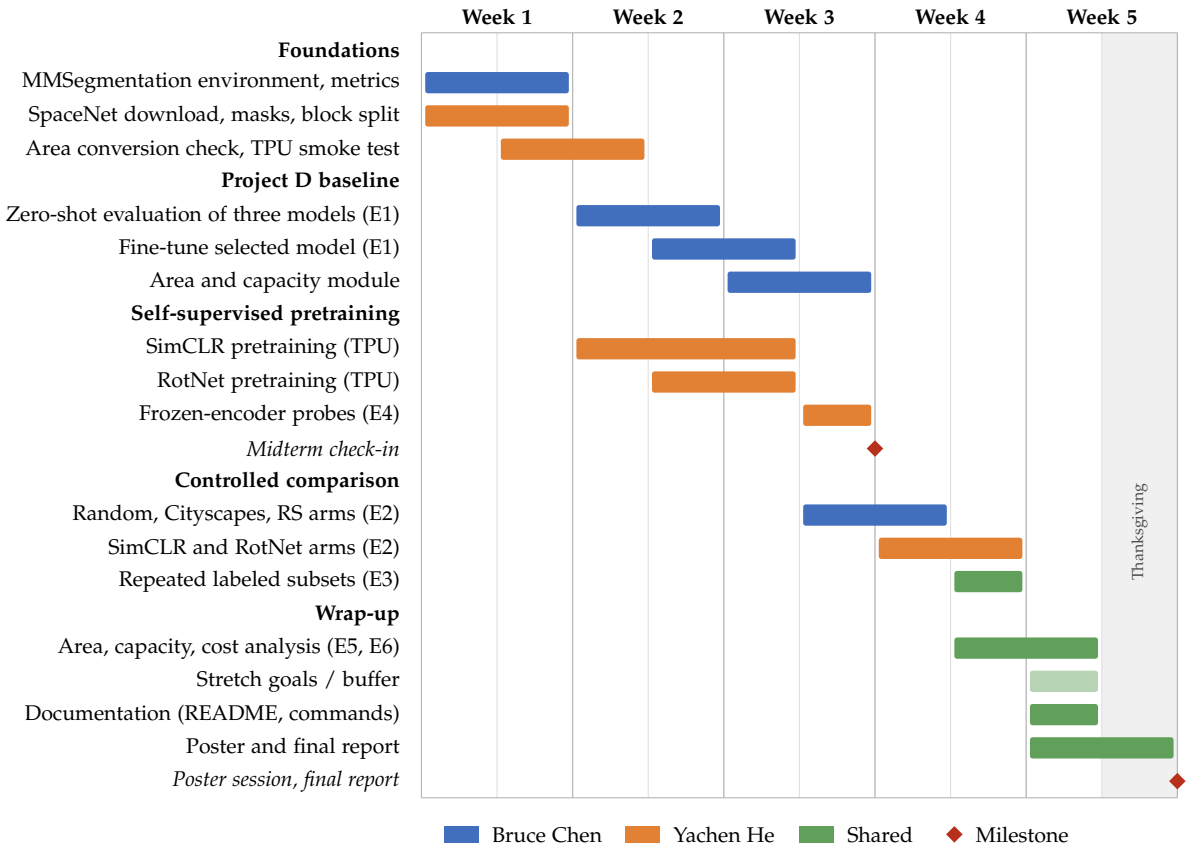


Figure 2: Five-week schedule (two columns per week). The shaded column marks Thanksgiving.

	Bruce Chen	**Yachen He**
Foundations	Pinned Colab MMSegmentation environment; metric code (IoU, precision, recall, bootstrap intervals)	SpaceNet download, mask rasterization, percentile stretch, geographic block split, statistics; TPU setup
Project D baseline	Zero-shot evaluation of the three checkpoints; model selection; full fine-tuning (E1)	Sample visualizations and dataset documentation; pixel-to-m^2 verification against polygons
Pretraining and probes	Encoder extraction from the Cityscapes and remote-sensing checkpoints; weight-loading checks for all arms	SimCLR and RotNet pretraining on TPU; frozen-encoder probes (E4)
Controlled comparison	Random, Cityscapes and remote-sensing arms at all budgets (E2); half of E3	SimCLR and RotNet arms at all budgets (E2); half of E3
Analysis	Area and capacity module; error analysis per evaluation area (E5); failure cases	Label-efficiency curves; probe analysis; compute and credit tracking (E6)
Writing	Report: baseline, area and capacity sections; half of poster	Report: data and pretraining sections; half of poster

Table 5: Balanced responsibilities (approximately 50% each).

H3. Remote-sensing transfer is the strongest initialization at small budgets, and SimCLR on unlabeled Las Vegas chips closes part of the gap between Cityscapes transfer and remote-sensing transfer.

H4. RotNet is the weaker self-supervised method: overhead images have no natural "up", so rotation can be predicted mainly from cues such as shadow direction, which say little about roof outlines; its probe (E4) shows this most clearly.

H5. Area errors come mainly from roof boundaries and shadows. With all labels, signed errors largely cancel in aggregate, while per-area absolute errors remain noticeably larger; at $n = 5\%$, area is systematically under- or over-estimated.

8.2 Success criteria: the mid-term and final exams

Targets below are set for our data size and training length; differences between arms are judged with the intervals of Section 3.6.

	Check	Target
Midterm (end of Wk 3)	Data pipeline	All Las Vegas chips converted; overlays checked on at least 20 random chips; no chip shared across partitions; rasterized area within 2% of polygon area on sample chips
	Project D baseline (E1)	Three checkpoints evaluated without fine-tuning; the selected model fine-tuned at $n = 100\%$ reaches roof IoU ≥ 0.70 on the validation blocks
	Pretrained encoders	SimCLR and RotNet losses converge; RotNet pretext accuracy reported; both load into DeepLabV3+ with no missing or unexpected encoder keys
	Area and capacity module	Exact on synthetic masks; reference area and capacity computed for every evaluation area
Final (end of Wk 5)	Core comparison (E2, E3)	All 15 (arm, budget) results on the test blocks with 95% bootstrap intervals; three subsets for $n \in \{5\%, 20\%\}$
	Label efficiency	IoU-versus-budget curve per arm; the smallest budget at which each pretrained arm matches random initialization at $n = 100\%$ within the intervals, or a clear statement that none does
	Area and capacity error (E5)	Signed and absolute area error (m^2, %) and capacity error (kWp) for every model, per evaluation area and in aggregate
	Cost (E6)	TPU chip-hours and A100-hours logged per run; spending within the TPU share and the Colab plan
	Deliverables	Code, configurations, logs, README with commands, figures, poster, report

Table 6: Mid-term and final "exams".

Final deliverables. (1) A clean code repository with data conversion, pretraining, fine-tuning and evaluation scripts; (2) training logs and loss curves for every run; (3) the comparison tables and IoU-versus-budget curves for E1–E6, with example predicted masks and failure cases such as shadows, tree cover and ambiguous roof boundaries; (4) a README documenting all settings and the exact commands; (5) the poster and the final report, including a statement of each member's contributions. A well-supported finding that no pretraining helps would also answer the research question.

AI assistance disclosure. An AI assistant (Claude) helped draft and organize this proposal from our project plan. During the project we plan to use the Gemini API through our course keys for coding assistance. The proposal applies the transfer-learning methods of Project D to overhead imagery and borrows the self-supervised comparison of Project 4. The team will verify dataset details, calculations, experimental claims and the final text before submission, and will separately disclose AI use in the final report.

References

[1] P. Gagnon, R. Margolis, J. Melius, C. Phillips, and R. Elmore. Rooftop solar photovoltaic technical potential in the United States: A detailed assessment. Technical Report NREL/TP-6A20-65298, National Renewable Energy Laboratory, 2016.

[2] J. Yu, Z. Wang, A. Majumdar, and R. Rajagopal. DeepSolar: A machine learning framework to efficiently construct a solar deployment database in the United States. *Joule*, 2(12):2605–2617, 2018.

[3] A. Van Etten, D. Lindenbaum, and T. M. Bacastow. SpaceNet: A remote sensing dataset and challenge series. arXiv:1807.01232, 2018.

[4] SpaceNet on AWS. Registry of Open Data on AWS. https://registry.opendata.aws/spacenet/, accessed October 26, 2026.

[5] J. Long, E. Shelhamer, and T. Darrell. Fully convolutional networks for semantic segmentation. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3431–3440, 2015.

[6] O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional networks for biomedical image segmentation. In *MICCAI*, pp. 234–241, 2015.

[7] MMSegmentation Contributors. MMSegmentation: OpenMMLab semantic segmentation toolbox and benchmark. https://github.com/open-mmlab/mmsegmentation, 2020.

[8] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam. Encoder-decoder with atrous separable convolution for semantic image segmentation. In *European Conference on Computer Vision (ECCV)*, pp. 801–818, 2018.

[9] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

[10] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, and B. Schiele. The Cityscapes dataset for semantic urban scene understanding. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3213–3223, 2016.

[11] J. Wang, Z. Zheng, A. Ma, X. Lu, and Y. Zhong. LoveDA: A remote sensing land-cover dataset for domain adaptive semantic segmentation. In *NeurIPS Datasets and Benchmarks Track*, 2021.

[12] F. Rottensteiner, G. Sohn, J. Jung, M. Gerke, C. Baillard, S. Bénitez, and U. Breitkopf. The ISPRS benchmark on urban object classification and 3D building reconstruction. *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, I-3:293–298, 2012.

[13] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. A simple framework for contrastive learning of visual representations. In *International Conference on Machine Learning (ICML)*, pp. 1597–1607, 2020.

[14] S. Gidaris, P. Singh, and N. Komodakis. Unsupervised representation learning by predicting image rotations. In *International Conference on Learning Representations (ICLR)*, 2018.

[15] O. Mañas, A. Lacoste, X. Giró-i-Nieto, D. Vázquez, and P. Rodríguez. Seasonal Contrast: Unsupervised pre-training from uncurated remote sensing data. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[16] Google Cloud. Cloud TPU pricing. https://cloud.google.com/tpu/pricing, accessed October 26, 2026.

[17] C. McCormick. Colab GPUs features and pricing (updated March 2026). https://mccormickml.com/2024/04/23/colab-gpus-features-and-pricing/, accessed October 26, 2026.